the program does. Tests that you can run every time you make a change (or even when you don’t make a change) to make sure that the program is working as it should. That sounds better than a late-night panic when things go wrong. To live in that world, you need to learn how to write tests.

10.2. How to write tests

Ask your dentist which teeth you should brush. The answer will be “only the ones you want to keep.” It’s the same with tests—test only the parts of the program that you want to work. There are three basic steps:

1. Write a test.
2. Watch it fail.
3. Change the program to make the test pass.

This sequence begins with *writing a test*. How do you write a test?

Frameworks

Although it’s common to use a *kitchen-sink-included* testing framework to write tests, these frameworks aren’t necessary to learn *how* to test. By learning to write tests without a testing framework, you’ll understand the principles behind testing frameworks.

A test can be just a stated requirement for what the program should do. A simple requirement for something the tracking application should do is this:

It should record each product the user clicks on.

If this test is translated into something that can be executed (a test program), then it can be run automatically against the real program every time a change is made. Tests that are run automatically provide frequent feedback that the program does what it should. Without tests, there’s only